

Проект: Наложение водяного знака на BMP изображение (C++ и Git/GitHub)

Описание задачи

В рамках командного проекта необходимо разработать программу на C++, которая накладывает цифровой водяной знак (второе изображение) на исходное BMP-изображение и сохраняет результат в новый BMP-файл. Проект выполняется группой из трёх студентов, используя систему контроля версий Git и платформу GitHub для совместной работы.

В ходе работы студенты создадут класс для работы с изображениями BMP, реализуют механизм наложения водяного знака и обеспечат автоматическую сборку проекта с помощью GitHub Actions. Результатом проекта будет публичный репозиторий на GitHub, содержащий исходный код программы, историю совместных коммитов, автоматизированную сборку (CI) и документацию в файле README.

Цели проекта

- Научиться разбивать большую задачу на части и работать в команде над общим кодом.
- Освоить практическое использование Git: создание локального репозитория, коммиты, слияние изменений, разрешение конфликтов, отправка изменений на GitHub.
- Получить опыт использования GitHub для совместной разработки: совместный репозиторий, Pull Request'ы (по желанию), код-ревью.
- Закрепить знание C++ в части работы с файловым вводом-выводом (бинарные файлы), создании и использовании классов, манипуляции с массивами байтов.
- Разобраться с форматом изображений BMP: чтение заголовка файла, пиксельных данных; научиться сохранять изменённое изображение в правильном формате.
- Реализовать простой алгоритм наложения водяного знака на изображение (визуальное наложение, чтобы исходное изображение оставалось видимым).
- Настроить простейший CI (Continuous Integration) с помощью GitHub Actions для автоматической сборки и тестирования проекта при обновлениях репозитория.
- Оформить документацию в файле README, описав работу программы, структуру проекта, инструкции по сборке и использованию, а также результаты тестовых запусков.

Распределение задач по ролям

Каждый участник команды берёт на себя определённую часть работы. Распределение обязанностей между студентами следующее:

1. Студент 1 – Разработка классов для работы с BMP-файлами:

- Создает класс (или классы) на C++, описывающий BMP-изображение. Этот класс должен уметь читать BMP-файл с диска и сохранять изображение в файл BMP.
- Предусматривает в классе поля для хранения ключевой информации о изображении: ширина, высота, битовая глубина (цветность), а также массив пикселей изображения.
- Реализует методы для чтения и записи BMP. Необходимо открыть файлы в бинарном режиме и правильно считывать/записывать заголовок и пиксельные данные. Формат BMP подразумевает наличие заголовка файла (BITMAPFILEHEADER) и информационного заголовка (BITMAPINFOHEADER), за которыми следуют сами пиксели. Для упрощения можно ограничиться форматом 24-бит BMP без сжатия (Windows Bitmap v3, с заголовком BITMAPINFOHEADER). В таком формате каждый пиксель хранится 3 байтами (Blue, Green, Red), а каждая строка выравнивается по 4 байта.
- Убеждается, что класс корректно загружает данные: считывает размеры изображения, цветовую глубину, а затем массив пикселей. Важно: в BMP-файлах пиксели зачастую хранятся в порядке “bottom-up” (нижняя строка изображения идет первой в данных). Необходимо обработать это, чтобы при наложении водяного знака изображение не оказалось перевернутым. Также, стоит учесть, что в 24-бит BMP каждая строка дополняется пустыми байтами (padding) до кратности 4 байтам – эти байты не содержат цветовых данных и их нужно пропускать при чтении.
- Записывает реализацию на C++ (можно использовать struct для заголовков или загрузить данные побайтово с помощью std::ifstream с флагом std::ios::binary). Внешние библиотеки для работы с изображениями использовать нежелательно – рекомендуется реализовать парсинг BMP самостоятельно, чтобы лучше понять формат.
- Покрывает код комментариями, объясняя ключевые шаги (например, чтение заголовка, вычисление смещения до пикселей и т.д.).
- После отладки функциональности чтения/записи BMP, студент 1 загружает свой код в общий репозиторий (например, в основную ветку или отдельную ветку) для доступа остальными участниками.

2. Студент 2 – Реализация наложения водяного знака:

- Разрабатывает функцию или метод, реализующую наложение водяного знака (второго изображения) на основное изображение.
- Входные данные для этой части: основной BMP-файл (оригинальное изображение) и файл с водяным знаком (также BMP). Оба изображения можно загрузить с помощью классов, созданных студентом 1.
- Требуется продумать алгоритм визуального наложения, при котором содержимое исходного изображения остаётся видимым, а водяной знак проявляется поверх него. Например, можно реализовать полупрозрачное наложение: для каждого пикселя исходного изображения и соответствующего пикселя водяного знака вычислить новый цвет как смесь обоих. Самый простой способ – усреднение цветовых компонентов:
 - $\text{new_red} = (\text{orig_red} + \text{mark_red}) / 2$
 - $\text{new_green} = (\text{orig_green} + \text{mark_green}) / 2$
 - $\text{new_blue} = (\text{orig_blue} + \text{mark_blue}) / 2$

Такой подход даст видимый водяной знак, при этом фон (оригинал) тоже просматривается. Примечание: можно выбрать другое соотношение, например 70% оригинального цвета и 30% от цвета метки, чтобы водяной знак был более прозрачным. В общем случае: $\text{new_color} = \text{orig_color} * (1 - \alpha) + \text{mark_color} * \alpha$, где $\alpha = 0.3\text{--}0.5$ (30–50%).

- Если изображение водяного знака по размеру меньше основного изображения, нужно решить, куда его разместить. Можно, например, наложить метку в правом нижнем углу исходной картинке либо по центру. Для простоты допустимо требовать, чтобы водяной знак имел такие же размеры, как и основное изображение (тогда накладываются пиксель-в-пиксель). Если размеры различаются – реализуйте аккуратное наложение: переберите пиксели области, соответствующей водяному знаку, и измените только эту область основного изображения.
 - Студент 2 также должен использовать разработанный класс BMP от студента 1: загрузить оба изображения в объекты класса, выполнить наложение, получить новый пиксельный массив результата. Затем с помощью метода сохранения записать новый BMP-файл с водяным знаком.
 - Разрабатывает небольшие тестовые примеры для отладки: например, проверить работу на двух маленьких изображениях (5x5 или 10x10 пикселей) вручную, убедиться, что алгоритм правильно смешивает цвета, и что размеры выходного изображения корректны.
 - Пишет чистый, понятный код, снабжает его комментариями (объясняя, например, выбор коэффициента смешения, обработку граничных случаев типа выход за границы, если такое возможно).
 - После завершения этой части студент 2 добавляет свой код в репозиторий. Важно: при интеграции убедиться, что не затёрт код студента 1. Желательно выполнить слияние (merge) или rebase своих изменений с кодом студента 1. Если ведётся работа в разных ветках, студент 2 может открыть Pull Request для своего функционала и убедиться вместе с коллегами, что всё слилось корректно.
3. Студент 3 – Сборка проекта, интеграция и инфраструктура:
- Берёт на себя роль интегратора проекта. Сначала проверяет работу, выполненную студентами 1 и 2: ревьюит код на наличие ошибок, проверяет, правильно ли реализовано чтение/запись BMP и алгоритм наложения. По необходимости вносит исправления или предлагает их авторам.
 - Реализует основную функцию (main) программы, которая связывает все компоненты воедино. Главная функция должна обеспечивать:
 - Чтение имён входных файлов (можно через параметры командной строки argv или простым вводом с консоли). Требуется получать путь к исходному изображению, путь к файлу водяного знака и путь для выходного файла.
 - Загрузку BMP-файлов в объекты класса изображения (используя функциональность студента 1).
 - Вызов функции наложения водяного знака (разработанной студентом 2), получая на выходе изменённое изображение с водяным знаком.
 - Сохранение результата в новый BMP-файл на диске.
 - Вывод пользователю понятных сообщений о ходе выполнения, либо сообщений об ошибках (например, «Файл водяного знака не найден» или «Размеры изображений не совпадают» – эти проверки желательно предусмотреть).

- Обеспечивает структуру проекта: например, настроить сборку с помощью CMake или другого инструмента (по согласованию с преподавателем), организовать файлы по папкам (если необходимо: отдельные директории для заголовков, исходников, тестов).
- Настраивает непрерывную интеграцию (CI) с помощью GitHub Actions. Для этого студент 3 должен создать файл workflow (например, `.github/workflows/c-cpp.yml`), который автоматически собирает проект при каждом пуше/мерже в основную ветку. Достаточно настроить сборку под одной платформой (например, Ubuntu latest с компиляцией g++). Если команда использует CMake, Action должен запускать шаги `cmake ..`, `cmake --build` и т.д. Если сборка без CMake – можно напрямую вызывать компилятор (g++/clang) для файлов проекта. Цель: убедиться, что после каждого коммита проект успешно компилируется, и не допустить, чтобы в репозиторий попал неработающий код.
- При наличии тестов (например, функция, проверяющая корректность наложения на известных пикселях), настроить CI на их запуск и вывод результатов. Например, можно написать небольшой unit-тест: создать в коде два искусственных изображения (например, одно залить зелёным, другое – красным небольшого размера), наложить и проверить несколько пикселей результата на ожидаемые значения. Эти тесты можно запускать в Actions для автоматической проверки логики.
- После успешной интеграции всех компонентов студент 3 формирует финальную структуру репозитория: убеждается, что добавлены все исходники, бинарные тестовые файлы (если нужны), игнорируются лишние файлы (например, использовать `.gitignore` для локальных сборочных файлов). Также проверяет, что в репозитории нет конфликта и история коммитов отражает вклад каждого участника.
- Отвечает за написание подробного README-документа в корне репозитория. В README должны быть описаны: постановка задачи, кратко — как устроено решение, кто какие части выполнил (опционально), как собрать проект и запустить программу, примеры запуска, а также описание тестовых сценариев и, при необходимости, результаты работы (например, ссылка на полученное изображение с водяным знаком). Документация должна быть структурирована (разделы, заголовки, списки) и понятна любому, кто зайдёт в проект.

Требования к реализации

- Язык и стандарты: использовать C++ (рекомендуемый стандарт – C++17 или новее). Соблюдать правила кодирования, писать понятный и читаемый код. Желательно разнести логику по функциям и классам, избегая слишком длинных функций.
- Работа с файлами: для чтения/записи BMP использовать файловые потоки C++ (`std::ifstream / std::ofstream`) с флагом `std::ios::binary`. Убедиться, что файлы открылись (проверять `ifstream.is_open()`), и что размер прочитанного соответствует ожидаемому. Обработка ошибки открытия файлов (неправильный путь, отсутствие доступа) с выводом сообщений.
- Формат изображения: как отмечалось, ограничиться несжатым 24-битным BMP. Это значит:

- Заголовок файла 14 байт (поля: сигнатура “BM”, размер файла, зарезервировано, смещение пиксельных данных) и заголовок BITMAPINFOHEADER 40 байт (размер структуры, ширина, высота, число color planes, бит на пиксель = 24, компрессия = BI_RGB (0), размер изображения может быть 0 для несжатого, разрешение, количество цветов и важные цвета) . В итоге пиксельные данные начинаются со смещения, указанного в bfOffBits (обычно 54 байта от начала файла для 24-бит BMP).
- Пиксельные данные: для каждого пикселя хранятся три байта (B, G, R). Порядок пикселей по умолчанию: строки в обратном порядке (нижняя строка – первая в данных, верхняя – последняя), если высота в заголовке положительная . Можно считать, что высота будет положительная (стандартный случай). Каждая строка дополнена padding-байтами до кратности 4. Например: для изображения шириной 3 пикселя (9 байт данных) нужно добавить 3 байта заполнения, чтобы суммарно строка заняла 12 байт (кратно 4). Студенты должны учесть пропуск этих заполнителей при чтении и записи.
- Без использования готовых библиотек для BMP: реализовать разбор заголовков вручную (через чтение двоичных данных в структуры или через побайтовое чтение). Нужно протестировать чтение и запись на небольших изображениях.
- Алгоритм наложения: должен корректно работать для всех пикселей накладываемой области. Убедиться, что не происходит переполнения при сложении цветовых компонент (например, складывать в переменном типе, способном вмещать сумму двух байтов, или приводить к int перед сложением). После вычисления усреднённого значения привести результат обратно к uint8_t (диапазон 0-255).
- Совместная разработка с Git:
 - Каждый студент должен работать с локальным репозиторием, регулярно фиксировать изменения (частые коммиты с осмысленными сообщениями).
 - Команда должна использовать общий удалённый репозиторий на GitHub. Можно создать организацию или просто один из студентов создает репозиторий и дает доступ остальным.
 - Разработка может вестись в нескольких ветках (например, file-reader, watermark-algo, integration), которые затем сливаются. Практика использования Pull Request приветствуется – это позволит не только интегрировать код, но и провести код-ревью друг друга.
 - В истории коммитов репозитория должны прослеживаться вклад каждого участника. Не допускайте ситуации, когда весь код добавлен одним коммитом от одного человека. Продемонстрируйте командную работу: например, студент 1 делает коммит с классом BMP, студент 2 позже – коммит с функцией наложения, студент 3 – коммиты по интеграции и CI. Допустимы и совместные правки, но старайтесь разграничить зоны ответственности.
- CI/CD – автоматическая сборка: необходимо настроить простой Continuous Integration с помощью GitHub Actions:
 - Добавьте конфигурационный файл workflow (YAML) в папку .github/workflows/. Например, можно назвать файл ci.yml. В нём определить действие, выполняющееся при событиях push и pull_request в основной ветке.
 - В задаче (job) CI как минимум должен быть шаг сборки проекта. Например, использовать образ ubuntu-latest и шаги:

- Установка соответствующего компилятора (Ubuntu по умолчанию имеет g++, можно использовать его).
 - Если используете CMake: шаги генерации и сборки через CMake.
 - Если без CMake: прямая компиляция командой, например, `g++ -std=c++17 -Wall -Wextra -O2 -o watermark main.cpp bmp.cpp utils.cpp`.
- При наличии тестов, добавить шаг запуска тестов (например, выполнение скомпилированной программы на подготовленных входных файлах и проверка кода возврата, либо выполнение отдельных unit-тестов).
- Результатом работы Actions должен быть статус: успешно собирается и (если предусмотрены) проходит тесты. Студент 3 может добавить бейдж статуса сборки в README.md.
- Документация и стиль кода:
 - Репозиторий должен содержать файл README.md, в котором описана поставленная задача и приведены инструкции для пользователей проекта. Оформите README в стиле Markdown: используйте заголовки, списки, при необходимости добавляйте иллюстрации или схемы. Обязательно укажите, как собрать проект (например, требуемая версия компилятора, команды сборки) и как запустить программу, приведите пример запуска с пояснениями.
 - Код следует снабдить комментариями там, где логика может быть неочевидна. Комментарии должны отражать суть происходящего, но не избыточно (избегайте комментариев к очевидным вещам).
 - Именованье переменных, функций и файлов – в едином стиле (желательно придерживаться принятого стиля кодирования C++: например, CamelCase для классов, snake_case для функций/переменных, понятные имена). Код должен быть максимально читаемым и аккуратным.
 - В репозитории не должно быть временных файлов сборки, исполняемых файлов и т.п. – используйте .gitignore.
- Тестирование: Кроме ручного тестирования (открыть полученный BMP в просмотрщике изображений и визуально убедиться в наличии водяного знака), подготовьте автоматизированные тесты насколько это возможно:
 - Минимум – включите в репозиторий пару тестовых изображений (например, original.bmp и watermark.bmp небольшого размера) и ожидаемый результат expected.bmp. В README опишите, что должно получиться при наложении, и приложите (или опишите) результат.
 - Возможно, реализуйте модульные тесты основных функций (например, функция blendPixels(orig, mark) должна правильно рассчитывать усреднённое значение цвета – можно протестировать эту логику отдельно).
 - При наличии time, можно сделать обратную проверку: например, наложить водяной знак, затем сохранить файл, загрузить его снова и проверить, что размер и заголовок соответствуют ожидаемым, а пиксели изменены именно в области наложения.
 - В идеале, все ключевые компоненты (загрузка BMP, сохранение BMP, наложение) должны быть проверены. Стремитесь к тому, чтобы в README или в виде автоматических тестов было показано, что программа работает правильно.

Для успешного выполнения задания рекомендуем такую последовательность шагов:

1. Подготовка репозитория: Один из студентов (или вместе, за одним компьютером) инициализирует новый Git-репозиторий. Создайте на GitHub пустой репозиторий (например, bmp-watermark-project) и склонируйте его локально. Добавьте файл README.md с названием проекта и базовым описанием. Сделайте первый коммит и опубликуйте (push) на GitHub. Дайте доступ репозитория остальным участникам (через добавление Collaborators или сделайте приватный организационный репозиторий).
2. Проектирование решения: Все вместе обсудите и спланируйте архитектуру программы:
 - Определите, какие классы понадобятся. Очевидно нужен класс ImageBMP (название условное) с методами load(filename) и save(filename). Возможно, имеет смысл сделать отдельный класс для водяного знака или функцию applyWatermark(baseImage, markImage).
 - Решите, как будете смешивать пиксели (выберите коэффициент прозрачности или алгоритм).
 - Продумайте, как будете проверять корректность (какие тестовые файлы нужны, как организовать вывод).
 - Запишите краткий план, распределите, кто и как реализует свою часть, в каком порядке будут интегрироваться изменения.
3. Реализация класса BMP (Студент 1): Начните с разработки класса для работы с BMP:
 - Создайте ветку bmp-class-dev (или назовите по-другому) в репозитории для этой задачи.
 - Реализуйте чтение заголовка: можно создать #pragma pack(push, 1) структуры для BITMAPFILEHEADER и BITMAPINFOHEADER по спецификации и просто считать их из файла методом read(). Либо читать поля по отдельности. Учтите порядок байтов (в BMP little-endian, но на Intel CPU это совпадает с порядком в памяти, так что можно читать напрямую).
 - Реализуйте чтение пиксельного массива: выделите буфер нужного размера (например, вектор байт или new uint8_t[...]), прочитайте построчно, пропуская padding. Удобно читать построчно: ширина*3 байт реальных данных + нужное количество байт игнорировать (padding) перед чтением следующей строки.
 - Аналогично, реализуйте сохранение: записать сначала оба заголовка (не забыв обновить поля, например bfSize – общий размер файла, biSizeImage – размер массива изображения в байтах или 0 для BI_RGB несжатого), затем записать пиксели построчно, добавляя необходимый padding в файл.
 - По ходу разработки проверяйте метод load/save на простых примерах. Можно создать вручную небольшой BMP (или найти готовый) 2x2 пикселя для отладки.
 - Закоммитьте изменения и откройте Pull Request (если используете) или просто пушьте в основную ветку, чтобы другие участники смогли использовать класс.
4. Реализация наложения водяного знака (Студент 2): После того, как класс BMP доступен:
 - Получите последние изменения (git pull либо примите PR).
 - В своей ветке (например, watermark-feature) реализуйте функцию наложения. Это может быть метод класса ImageBMP (например, ImageBMP::applyWatermark(const ImageBMP& mark)) или отдельная

функция, которая принимает два объекта ImageBMP и возвращает новый объект/структуру с результатом.

- Убедитесь, что размеры изображений совместимы (например, водяной знак не выходит за границы оригинала). Если выходит – можно пока ограничиться наложением по размеру пересечения или вывести ошибку.
- Реализуйте смешивание пикселей в цикле по высоте и ширине накладываемой области. Выполните расчёт новых цветовых значений по выбранной формуле. Учтите, что если изображения bottom-up, то итерироваться по массиву можно либо с конца, либо скорректировать индексы. Проще может быть в начале привести оба изображения к массиву пикселей в обычном порядке строк (top-down) – то есть перевернуть изображения в памяти – и затем накладывать.
- Протестируйте функцию на нескольких случаях. Например, если наложить изображение само на себя – результат должен быть тем же изображением (если $\alpha=0$ для метки) или чуть светлее (если 50/50). Если наложить чисто белое изображение как водяной знак с 50% прозрачности – исходное должно осветлиться наполовину. Если наложить чисто чёрное – исходное потемнеет.
- Отладьте, затем сделайте коммит с сообщением, описывающим реализованную возможность. Отправьте изменения в репозиторий.

5. Интеграция и завершающие работы (Студент 3): Получив компоненты 1 и 2:

- Обновите локальный репозиторий, убедитесь, что весь код на месте. Разрешите возможные конфликты, если изменения затрагивали одни и те же файлы.
- Создайте файл main.cpp (или другой, если уже есть) с функцией main. Реализуйте логику: parse аргументы (или std::cin для имён файлов), использование методов класса для загрузки, применение функции наложения, сохранение результата. Добавьте сообщения std::cout для пользователя, например: *“Loading base image...”*, *“Applying watermark...”*, *“Saving result as output.bmp...”*.
- Протестируйте программу целиком на реальных файлах: возьмите какое-нибудь BMP-изображение (например, фотографию) и небольшой BMP-файл с водяным знаком (например, логотип или текст). Запустите программу и проверьте полученный выходной файл в просмотрщике изображений. Убедитесь, что водяной знак правильно наложился (виден, позиция правильная, исходное изображение просматривается, цвета корректны).
- Реализуйте необходимую обработку ошибок: если входных аргументов нет – вывод подсказки по использованию; если файлы не открываются – сообщить об ошибке; если размеры несовместимы – тоже сообщить.
- Настройте GitHub Actions: добавьте в репозиторий workflow-файл. Можно воспользоваться готовыми шаблонами Actions для C/C++ проектов. Например, GitHub предлагает шаблон C/C++ CI при создании нового Action: он устанавливает среду, собирает с помощью make. Вы можете адаптировать его под ваш проект. Основные шаги:
 - actions/checkout@v3 – чтобы получить код репозитория.
 - Установить среду (если нужен конкретный компилятор или зависимости).
 - Шаг сборки: либо make, либо команды cmake, либо g++ (в зависимости от того, как вы организовали сборку).
 - (Опционально) Шаг запуска: например, запустить вашу программу на тестовом файле. Учтите, что в Actions нет GUI, но вы можете

- сохранить выходной файл и артефакты, хотя это избыточно.
 - Минимально – убедиться, что программа завершается успешно.
- Добавьте бейдж статуса: после первого прогона Workflow на странице Actions будет предложен Markdown-бейдж. Скопируйте его в README (например, в начало файла с кратким индикатором статуса сборки).
- Напишите README.md: постарайтесь подробно, но кратко описать проект. Структура README может быть примерно такой:
 - Название и назначение проекта.
 - Используемые технологии (C++, Git, GitHub Actions) и цели (чему научились).
 - Как запустить проект: требования (например, “требуется компилятор поддерживающий C++17”), инструкции сборки (шаги cmake/make или команду компиляции), инструкцию использования (пример команды запуска с тремя файлами).
 - Описание того, как работает наложение водяного знака (какой алгоритм смещения пикселей используется).
 - Команда проекта: укажите имена участников и кто что делал (это покажет распределение работы).
 - Результаты: приложите небольшие изображения (или ссылки) “до и после” – например, вложите original.bmp, watermark.bmp и result.bmp в репозиторий (или хотя бы их уменьшенные копии в формате PNG для README). В тексте README вставьте картинки или дайте ссылки, чтобы проверяющий мог увидеть, как выглядит выходное изображение с наложенным знаком.
 - Возможные улучшения: укажите, что можно было бы доработать при наличии времени (например, поддержка других форматов или более умный алгоритм водяного знака).
- Проверьте ещё раз, что репозиторий содержит все необходимые файлы, Actions собирается без ошибок, README полнота и грамотность.
- Сделайте финальный коммит и пуш с пометкой, что проект завершён (например, сообщение “Project complete: integration and documentation done”).

По завершении проекта в вашем распоряжении должен быть репозиторий на GitHub, содержащий:

- Исходный код C++ с разделением по файлам (заголовочные файлы .h/.hpp, файлы реализации .cpp). Код компилируется без ошибок и предупреждений, выполняется согласно заданию.
- Файл проекта для сборки (если использовался CMake, Makefile или инструкция по сборке).
- Готовая программа: в репозитории либо в релизах может быть выложен скомпилированный исполняемый файл (не обязательно, основное – чтобы при сборке получался рабочий exe или бинарь).
- Несколько примеров входных файлов (BMP изображения для тестирования) и соответствующие им результирующие файлы с наложенным водяным знаком. В репозитории можно завести папку examples/ или test/ для этих файлов.
- Автоматический workflow GitHub Actions, статус сборки должен быть зеленым (успешным) для последней версии кода. Наличие бейджа статуса в README приветствуется.

- README.md – детальная документация по проекту: описание, инструкция, авторы, примеры, и т.д. Документ должен быть отформатирован в Markdown, структурирован заголовками и списками для удобства чтения.
- История Git: множественные коммиты от всех трёх участников. Коммиты должны иметь осмысленные описания (на английском или русском языке, отражающие суть изменений). В идеале, каждая функциональная часть добавлена отдельным коммитом или серией коммитов, а не всё разом.
- Комментарии в коде: особенно в сложных местах (разбор файла, алгоритм наложения), а также краткие описания классов и функций (можно в формате Doxygen-style, но не обязательно).

Проект будет считаться успешно выполненным, если:

- Программа корректно читает BMP-файлы и создаёт новый BMP с наложенным водяным знаком. Полученный файл открывается стандартными средствами просмотра изображений и визуально соответствует ожиданиям (водяной знак присутствует, фон исходного изображения виден).
- Код организован качественно: разделён по файлам/классам, легко читается, нет глупых дублирований, учтены основные ошибки (программа не падает на несуществующем файле, даёт понятное сообщение).
- В репозитории прослеживается активность каждого участника, работа действительно велась совместно с использованием Git (а не одним человеком в последний момент).
- Настроена и работает CI-сборка на GitHub – это показывает умение автоматизировать рутинные процессы и обеспечивает уверенность в работоспособности кода.
- Документация позволяет понять, что сделано, и повторить запуск без дополнительного обращения к авторам.